

LEZIONE 10/09/2026

Intro sul corso - dove siamo:

Chiuso Express, ci stiamo spostando in orizzontale per vedere i database. Lezioni introduttive 10/09 e 15/09 saranno teoriche, prima di affrontare nuovi linguaggi e nuovi tool.

DATABASE INTRO

È un sistema tecnologico che permette di gestire una collezione organizzata di dati coerenti.

Le caratteristiche fondamentali sono:

- Permettere le operazioni di creazione, cancellazione, aggiornamento e modifica dei dati
- Offrire strumenti per la ricerca avanzata di dati all'interno della collezione
- Garantire il salvataggio permanente

DB caratteristiche:

- dati
- collezioni
- crud
- coerenza
- transazioni
- persiste? Per es. database in memory, per il caching per esempio, non persiste ma è comunque un db. Nella maggior parte dei casi però una delle funzioni fondamentali del db è la persistenza.
- relazionale / non relazionale?
- omogeneità strutturale (dei dati)?
- gestito? gestito vuol dire che esiste un software con un suo proprio linguaggio che mi permette di creare ed accedere a una base di dati (es. MySQL), non gestito è sostanzialmente ciò con cui abbiamo avuto a che fare finora. DB management system stanno qui (management system ci suggerisce proprio l'essere gestito).

Persistenza → dal client ci aspettiamo uno stato effimero, che muore quando il client viene riavviato, quindi deleghiamo la centralizzazione (molti client così possono attingere allo stesso stato condiviso) e sopravvivenza (persistenza) dello stato al server → poi però sappiamo che il server è di suo "stateless": dentro un controller non abbiamo memoria, ogni endpoint riceve ciò che deve sapere dalla chiamata e risponde (in mezzo non c'è memoria). La memoria viene quindi da un terzo strato che funge da memoria del server → il database.

Storia dei DB:

La necessità di gestire dati in maniera organizzata nasce con i primi mainframe.

Negli anni '70, Edgar Codd dell'IBM lavora su una formalizzazione (anche logico-matematica) di come poteva essere organizzato un database introducendo i database relazionali.

I database sono quindi tra i pezzi di software più maturi con cui un developer ha a che fare. Alcuni db sono precedenti ai sistemi operativi su quali li facciamo operare (es. le origini di PostgreSQL precedono le origini di Windows).

Da allora sono nati diversi software che utilizzano le regole relazionali per fornire le funzionalità di un Database Management System (MySQL, MS SQL Server, IBM DB2, ...)

Terminologia:

1. Un **database** è una collezione organizzata di dati coerenti.
2. **(R)DBMS - (Relational) Database Management System** è il mezzo tecnologico che permette di gestire la collezione di dati (il database)
3. Nella lingua parlata, il termine database viene utilizzato anche per riferirsi al DBMS
4. **NoSQL** - Sono una categoria di DBMS che non implementano le regole relazionali (es. MongoDB) → nota: SQL è un linguaggio (structured query language). I cd database noSQL sono in realtà “non solo SQL”, perchè spesso hanno dentro SQL ma anche potenzialmente noSQL.

Operazioni sui dati

Le operazioni sui dati vengono eseguite in **transazioni**:

transazione → una sequenza di operazioni (logica / blocco di codice) che, se eseguita in modo corretto, produce una variazione (di stato del nostro db).

In caso di successo, il risultato delle operazioni deve essere permanente o persistente.

In caso di insuccesso si deve tornare allo stato precedente all'inizio della transazione.

Questa caratteristica (se qualcosa va in errore, annulla l'intera transazione e non solo gli effetti del codice successivo all'errore) è fondamentale per ottenere la **coerenza** (caratteristica necessaria / non opzionale di un db).

Invarianti → stati possibili della nostra funzione (stati coerenti, che “hanno senso”). Sono di fatto delle “regole di coerenza”

ACID properties → Le transazioni in un RDBMS hanno le seguenti proprietà:

1. **Atomicità**: la transazione è indivisibile nella sua esecuzione e la sua esecuzione deve essere o totale o nulla, non sono ammesse esecuzioni parziali
2. **Coerenza**: non devono verificarsi contraddizioni (incoerenza) tra i dati archiviati nel DB.
 1. e 2. garantiscono coerenza e fanno funzionare le nostre transazioni, o meglio l'atomicità è una delle cose che difendono la coerenza, ma posso potenzialmente avere coerenza senza atomicità.
3. **Isolamento**: ogni transazione deve essere eseguita in modo isolato e indipendente dalle altre transazioni, l'eventuale fallimento di una transazione non deve interferire con le altre transazioni in esecuzione.

Ha a che fare con il fatto che un db possa soddisfare più richieste contemporaneamente.
4. **Durabilità**: (o persistenza): una volta che una transazione abbia richiesto un commit work, i cambiamenti apportati non dovranno essere più persi.

DATABASE RELAZIONALI

I DB relazionali organizzano i dati in tabelle.

Una tabella di un database è formata da righe (o tuple) e colonne.

Le colonne rappresentano la struttura, ossia le caratteristiche del dato da memorizzare. Ogni riga rappresenta un dato inserito.

In JSON, le righe sono gli oggetti (non annidati) e le colonne le proprietà. nota: per rappresentare oggetti / array annidati useremo le relazioni.

La tabella, le sue caratteristiche e vincoli sono l'elemento base del nostro "magazzino" di dati.

nota: nei database la definizione del tipo di dati, struttura e vincoli avvengono a monte, in una fase preliminare:

```
const table = {
  columns: [
    { name: 'id', type: 'number', constraints: ['unique'] },
    { name: 'name', type: 'string' },
    { name: 'performance', type: 'number' },
    { name: 'salary', type: 'number' },
  ],
  data: [
    [1, 'Gian', 1, 30000],
    [2, 'Gian', 1, 30000],
    [3, 'Gian', 1, 30000],
    [4, 'Gian', 1, 30000],
    [5, 'Gian', 1, 30000],
    [6, 'Gian', 1, 30000],
    [7, 'Gian', 1, 30000],
  ]
};
```

Per creare una tabella dobbiamo decidere la sua struttura, che comprende:

Nome Tabella

Colonne

Indici

La scelta di quale e quante colonne non è così banale come può sembrare:

tralasciando i discorsi di "normalizzazione"

(<https://www.guvi.in/hub/dbms-and-sql-tutorial/introduction-to-normalization/>), scegliere correttamente dal principio i dati che le colonne devono contenere è fondamentale per evitare costosi cambiamenti nel futuro.

Nota: nei database, la struttura è strettamente legata ai dati che sono presenti. Se cambio struttura, dovrò necessariamente anche convertire tutti i dati in memoria.

È importante scegliere:

- Nome della colonna → Chiaro e immediatamente comprensibile + Future-proof
- Rappresentazione del dato da inserire → Es. una data di nascita (17-02-1988, 17th Feb 88, ...)
- Tipo di dato → Con cui il database gestirà l'informazione

- Attributi → Particolari caratteristiche per ogni colonna

Tipi di dati nei database

I database relazionali supportano nativamente diversi tipi di dato che possiamo raggruppare in:

1. Numeri
2. Stringhe
3. Date
4. Altri meno utilizzati
 - a. Dati geometrici/geografici
 - b. Dati binary (BLOB) → BLOB ci permette per es. di mettere un file in un db

1. Numeri nei database

a. numeri interi

- TINYINT
Utilizza solo un byte per salvare numeri che vanno da -128 a 127.
Viene utilizzato ad esempio per salvare i valori boolean, proprio perché è il tipo di dato più piccolo possibile
- SMALL / MEDIUMINT
Occupano rispettivamente 2 e 3 bytes
- INT
Utilizza 4 bytes per rappresentare numeri compresi tra -2.147.483.648 e 2.147.483.647
- BIGINT
Corrisponde ad un INT con il doppio di bytes disponibili

b. numeri decimali

- FLOAT(I, D)
un numero con la virgola da 4 bytes
- DOUBLE(I, D)
un numero con la virgola da 8 bytes
- DECIMAL(I, D)
un double che gestisce gli arrotondamenti considerando i numeri dal punto di vista finanziario

I e D identificano:

- il numero di cifre totali (I)
- quante di queste ne vogliamo dedicare alla parte decimale (D).

Ad esempio, utilizzando un float(6,4), il numero 56,01528012 diverrà 56,0153 proprio perché potrà avere al massimo 6 numeri, di cui 4 dopo la virgola.

Nota: perchè ci interessa distinguere tra la dimensione? perchè in ambito DB abbiamo potenzialmente a che fare con tonnellate di dati e vogliamo evitare di riservare spazio eccessivo per immagazzinare un determinato dato. Stesso motivo per il quale la I dei decimali indica la grandezza totale, non solo il numero prima della virgola: io voglio sapere lo spazio che occupa e poi decidere sulla precisione rispetto ai decimali.

2. Stringhe nel database

Anche in questo caso ciò che cambia è la capacità di ogni singolo tipo:

- CHAR(numero)
permette di indicare la lunghezza esatta della stringa da rappresentare, con il limite di 255 caratteri, risparmiando 1 byte
- VARCHAR(numero)
permette di indicare la lunghezza massima della stringa da rappresentare, con il limite di 255 caratteri. Consuma 1 byte in più per la flessibilità
- TEXT
fino a 65535 caratteri - è usato ad esempio per le descrizioni prodotto
- MEDIUMTEXT
16mb
- LONGTEXT
4.2gb - usato per salvare porzioni di HTML o lunghi testi

CHAR VS VARCHAR

Questa tabella ci mostra la differenza di consumo tra CHAR e VARCHAR se inseriamo una stringa di 4 caratteri. Come si nota, se sappiamo che i valori della nostra colonna saranno esattamente 4, ci conviene impostare un CHAR(4) e risparmiare 1 byte! Altrimenti, sfruttiamo la flessibilità di VARCHAR.

Nota: Varchar consuma 1byte in più perchè deve gestire due informazioni sulla stringa (lunghezza reale e lunghezza massima).

Value	CHAR (4)	Storage Required	VARCHAR (4)	Storage Required
' '	' '	4 bytes	' '	1 byte
'ab '	'ab '	4 bytes	'ab '	3 bytes
'abcd '	'abcd '	4 bytes	'abcd '	5 bytes
'abcdefgh'	'abcd '	4 bytes	'abcd '	5 bytes

3. Date nel database

- DATETIME
permette di memorizzare data e ora (YYYY-MM-GG HH:II:SS)
- DATE
contiene solo la data (YYYY-MM-GG)
- TIME
contiene solo l'orario (HH:II:SS)
- YEAR
contiene solo l'anno (YYYY)
- TIMESTAMP
può avere diversi formati

Primary Key → chiave primaria / identificatore della entry / dato

Ogni tabella deve avere una colonna adibita ad identificatore, cioè un valore diverso per ogni riga inserita, che permetta di individuarla univocamente. Questo vuol dire che attraverso una primary key potrò sempre risalire al record all'interno di una tabella.

Il caso più semplice e comune è avere una colonna id che contenga un numero intero che cresce automaticamente all'inserimento di ogni nuova riga

In rari casi, un identificatore può essere anche una coppia di due o più colonne → nota: noi non gestiremo per ora questa casistica, in quanto introduce della complessità: avere una chiave composta significa avere un vincolo di unicità su più colonne.

Attributi delle colonne di un DB

Ad ogni colonna, possiamo assegnare degli attributi:

- **NULL / NOTNULL**

indica che la colonna può o non può contenere il valore NULL.

Se una colonna è NOT NULL e durante il salvataggio non viene passato alcun valore per quella colonna, verrà restituito un errore e l'intera riga non verrà inserita.

Esempio:

In un database di un'anagrafe, la colonna data_di_nascita sarà NOT NULL per indicare che non è possibile salvare nel database di una persona senza indicare anche la data di nascita

- **DEFAULT**

permette di settare un valore di default alla colonna qualora non fosse presente alcun valore al momento del salvataggio / inserzione.

Esempio:

Se vogliamo contare il numero di volte in cui un prodotto è stato acquistato, può avere senso avere DEFAULT(0)

- **AUTO_INCREMENT**

Il valore della colonna è incrementato ad ogni nuovo record, per questo motivo è utilizzabile solo con le colonne di tipo numero.

Viene utilizzato ad esempio per gli ID: ogni volta che si aggiunge una nuova riga, in automatico l'ID sarà +1 rispetto alla riga precedente. In realtà, è ancora più corretto dire che auto_increment assegna un valore incrementato di uno rispetto all'ultimo valore mai assegnato a quella colonna (importante capire la differenza, perchè vuol dire che una cancellazione non permette di attribuire un id che era già stato occupato, anche se quel record è stato cancellato).

- **UNIQUE**

Indica che i valori di quella colonna devono essere unici, ossia non ci può essere alcun valore ripetuto all'interno della colonna.

La colonna primary key hanno di default gli attributi NOT NULL e UNIQUE, spesso hanno anche AUTO_INCREMENT